IN2011 Computer Networks : Practical 1

January 29, 2026

This practical session has several parts. One part will get you set us with the Azure virtual lab. One part will teach you the following core network programming skills:

- Understanding the different roles of *client* and *server* that are used in some protocols.
- Creating and using Java `Socket` and `ServerSocket` objects.
- Deadlocks and the challenges of reacting to server sources of input.

The last part is a *Coursework Challenge* that will give you one "flag" for coursework 2.

1 Discussion

What is an algorithm? What is a protocol? How are they different? What is a protocol stack? How do they help design and understanding?

2 Keeping Yourself Safe

As part of joining the University, you will have signed an agreement to follow the University rules and regulations. These include quite a few rules relating to networks. Have a look at:

`https://studenthub.citystgeorges.ac.uk/student-support-services/it-services/it-policies`

and see if you can answer the following:

- Can you use the University network to download material that endorses discrimination on the basis of religious belief?
- Can you use a personal e-mail address for official University business?
- What kind of information can you keep on an unencrypted device?

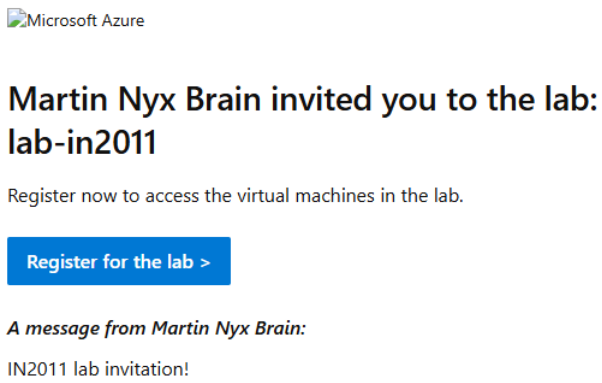
3 Getting Started With The Azure Virtual Lab

So that you can get practical experience working with a live network, we have set up a virtual lab. Every student has access to their own computer running on this lab. The virtual lab computers should be accessible from anywhere on the Internet and at any time. The amount of running time you have is limited but if you run into problems, please contact Martin Nyx Brain.

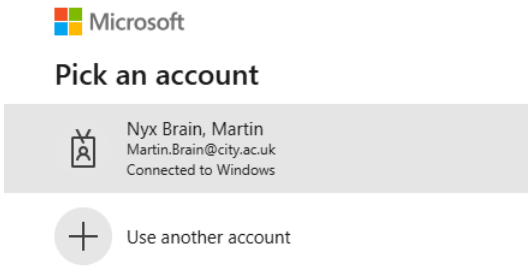
These instructions will get you set up:

1. You will need to have an RDP client installed. Windows machines have Microsoft Remote Desktop installed by default. Apple users will have to install it from `https://apps.apple.com/us/app/windows-app/id1295203466?mt=12`. Linux users will have to install `rdesktop`.

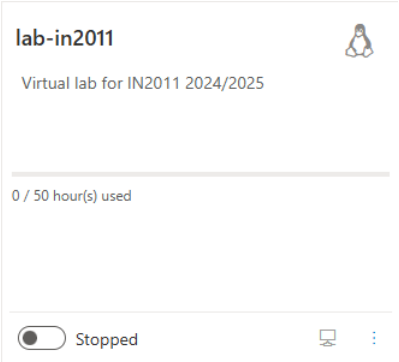
2. You should have received an e-mail invitation from "Microsoft Azure" with a link saying "Register for the lab":



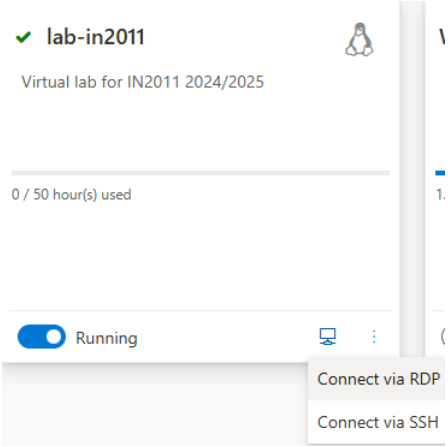
3. When asked to pick an account, choose your City account. If you are not logged in, you may have to authenticate:



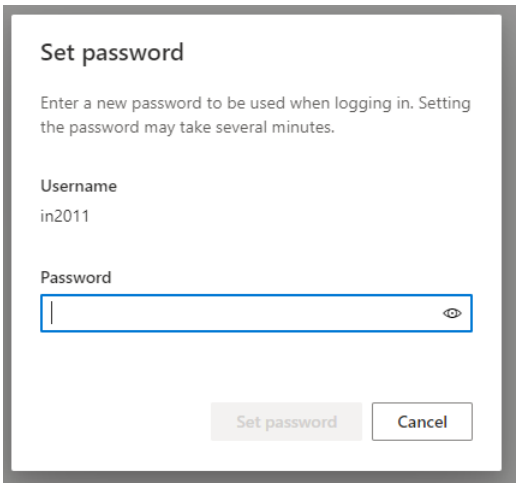
4. The main Azure Labs window should have a single box for your virtual machine. The toggle in the bottom left powers the computer on or off. Each user has 50 hours of compute time so remember to turn your computer off after you have finished working on it! Power the computer on, this may take a minute or so.



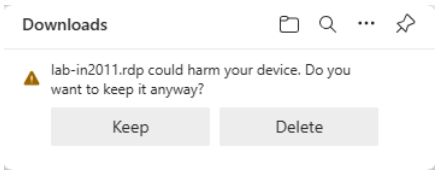
5. Towards the bottom right corner of the box there is an icon of a screen. Click on it and select "Connect via RDP":



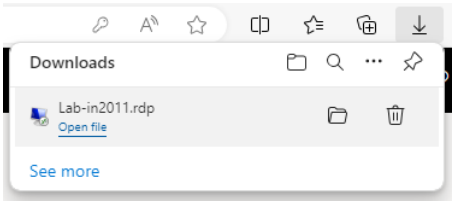
6. You will be required to set a new password for your lab computer. Like all password – please don't share this. Setting the new password may take a minute or so. You will only have to do this once when you first use the computer.



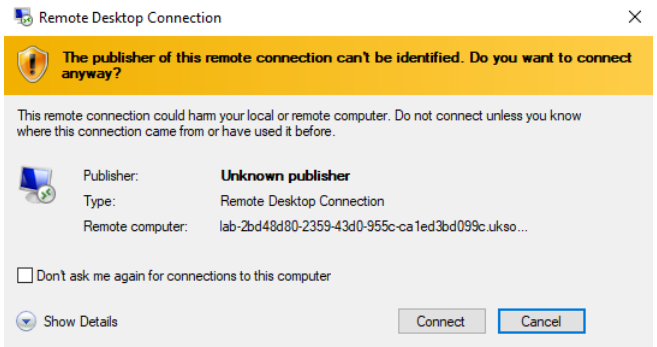
7. Select "Connect via RDP" again and it will download a file with the details of how to connect to the computer. Note that some browsers may mark this as potentially harmful.

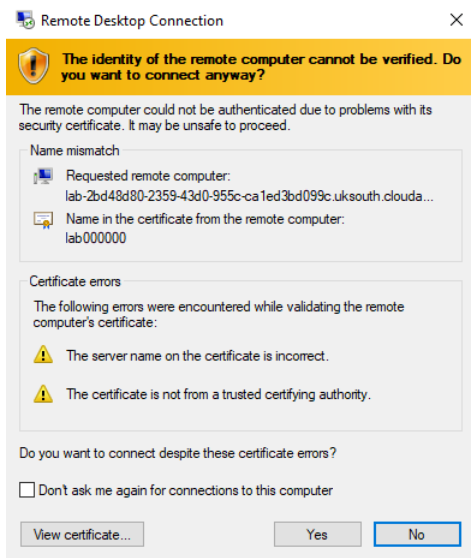


8. Open the file to start the remote desktop connection.

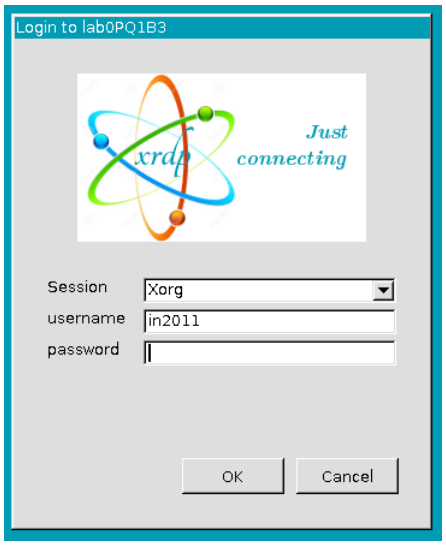


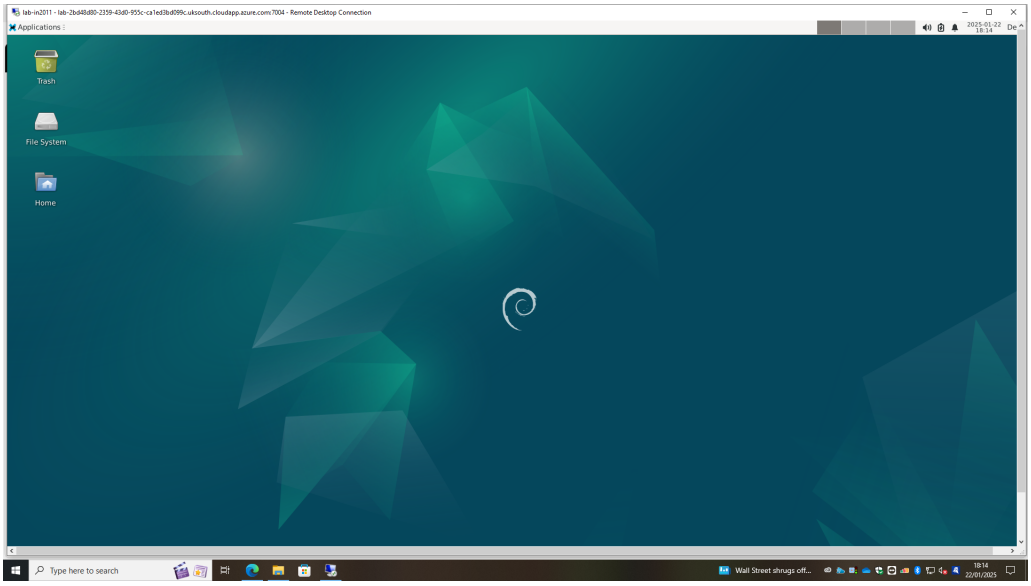
Again, you may get some security warnings. If they look like the ones below then you can safely ignore them:



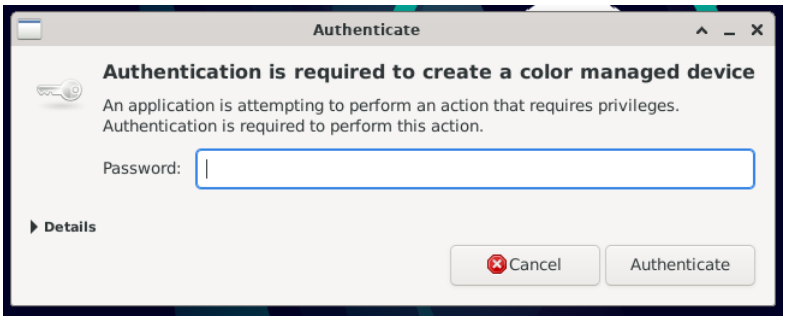


9. When you get a log in prompt, use the password you just set. This may take a minute or so. Once you have logged in you will see a Linux desktop.





10. In some circumstances you may get an authentication prompt about "color managed device". You can safely cancel this.



There are a number of ways to get data on and off the virtual lab computers.

1. The most professional option is to set up a private git repository (please do make sure it is private!), push the code from the machine you are working on, then clone a copy of the repository on the test machine and pull down the changes. There are many git tutorials on how to do this.

2. Many RDP clients let you share part of the client computer's filesystem with the remote computer. The set up is a little different depending on which RDP client you use but again, there are many tutorials available online showing how to do this.

3. As well as using Remote Desktop (RDP), you can also access the lab machines via Secure SHell (SSH). On the control panel, one of the options is to give the DNS address and port number needed to contact the machine via SSH. SSH also has a file transfer mechanism called SCP. Linux and Mac users will likely have this installed already. Windows users may need to install PuTTY or another SSH implementation. You can then directly copy files and directories to and from the lab machines.

4. If you have any kind of shared storage with a web front-end, such as One Drive, Dropbox or similar then you can copy the files there, connect to the lab machine, start a web browser and access them there.

5. If you get really really desperate, you can always e-mail them to yourself, connect to the lab machine, log into the web front-end for your e-mail and then save them from the sent mail!

Your virtual lab machine should now be set up and ready to use!

4 Network Programming

This section can be done on your Azure virtual machine or on your own computer.

4.1 Understanding the Terminology

The following paragraph explains the `Socket` objects in Java. Make sure that you understand, can define and give an example of all of the terms that are given *in italics*. Go back to your notes from the lecture or the reading if you are not sure.

> The Java standard library contains the *class* `java.net.Socket`. Each `Socket` *object* provides a network *service* which allows a *stream* of bytes to be sent to another program (maybe on a different computer) and a stream of bytes to be received from that program. This is similar to reading reading and writing to a file. Anything that is written will be sent to the other program and anything that program writes can be read. To do this `Socket` uses an *implementation* of TCP. TCP is a *connection-orientated transport layer protocol* which creates a *reliable* stream of bytes between two programs. To create a connection a one computer must act as a *server* and the other must act as a *client*. Once the connection is established both computers are equal.

4.2 Reading and Writing Data

Download `TCPClient.java` and `TCPServer.java` from Moodle. Read through them and make sure you understand what they do. Compile them, start `TCPServer.java` running and then run `TCPClient.java` while the server is running. What do you see?

Edit the two files to try out various different orders of read and write. For example, both write first then read, both read first then write, etc. What happens? Look up the `ready()` method and see if it helps fix any problems you encounter.

Edit them to send multiple lines of text. You may want to use a loop to read or write multiple times. Run the pair of programs multiple times. Do you always get the same text in the same order?

4.3 Port Numbers

We will cover port numbers in detail in lecture 5. For now it is enough to understand that they allow multiple different TCP connections between the same computers. This means you can listen to music and browse the web at the same time!

Try changing one or both port numbers. What combinations of port number work?

4.4 IP addresses

We will cover IP addresses in detail in lecture 4. For now it is enough to understand that they allow you to contact different computers. `127.0.0.1` is

a special address which refers to "The computer this program is running on". This is how you can communicate between two programs on the same machine.

Modify `TCPClient.java` to connect to the IP address `104.18.26.120`, the port number `80`, to send the string:

```
"GET / HTTP/1.1\r\nHost: www.example.com\r\n\r\n"
```

and to print out all of the response received. What does this look like?

Use a web browser to visit `http://www.example.com/` and choose "View Source" (different browsers have this option in different places, try right-clicking). Does this give you something similar? We will return to this in lecture 6.

If you work with another student you may be able to use `TCPClient.java` and `TCPServer.java` to send information between the two of you. If one runs the server and the other modifies their `TCPClient.java` to connect to the other person's IP address, then it *should* work. As we will see in later lectures there are a number of reasons why it might not work. Try to make sure:

- You are both on the same network.
- You use your operating system to find your IP address, not a website.
- You pick a port that is not affected by a firewall on either machine.

4.5 Multiple connections

`TCPServer.java` only accepts one connection and only creates one `Socket` object. Modify it to create two objects by calling `accept()` and to send anything that is read from the first `Socket` object to the second `Socket` object. You will now need to run two copies of `TCPClient.java` at the same time. What do you see?

Using `ready()` or threads you can extend this so that `TCPServer.java` relays anything it reads from any `Socket` to all other `Sockets`. This will create a rudimentary chat application.

5 Coursework Challenge

On your Azure virtual lab computer create and run a Java program that uses TCP to connect to IP address `10.200.51.18` or `10.200.51.19` port `2011` and reads anything the server sends.